
Práctica N° 9: TRANSFORMACIÓN 2D — SHEARING

1. OBJETIVO

- Entender los conceptos geométricos de deformación (shearing).
- Implementar y aplicar transformaciones de deformación en un entorno de gráficos moderno.

2. BASE TEÓRICA

2.1. Deformación (Shearing)

La operación de deformación o corte (*shearing*) en 2D produce una distorsión de la forma del objeto al estirarlo con respecto a un eje determinado (Figura 1).

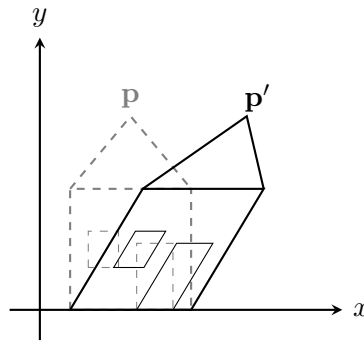


Figura 1: Ejemplo de deformación horizontal aplicada a una figura.

2.2. Deformación a lo largo del eje X

La deformación se realiza a lo largo del eje x aplicando las siguientes ecuaciones matemáticas:

$$\begin{aligned}x_2 &= x_1 + sh_x \cdot y_1 \\ y_2 &= y_1\end{aligned}$$

Donde sh_x es un factor que controla el nivel de deformación a lo largo del eje x . Este factor puede ser mayor o menor que 0. El efecto se muestra en la Figura 2.

En forma matricial homogénea de 3×3 (o su equivalente no homogénea de 2×2), esta operación se expresa como:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & sh_x \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

Cabe destacar que todos los puntos sobre el eje x (es decir, con coordenada $y = 0$) no se ven afectados por la matriz de transformación.

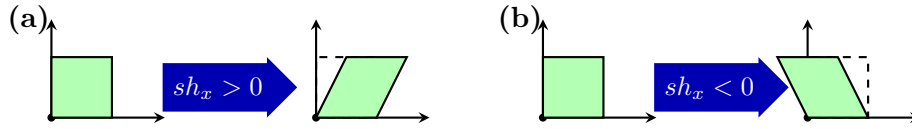


Figura 2: Representación gráfica de la deformación en el eje x (sh_x).

2.3. Deformación a lo largo del eje Y

La deformación paralela al eje y se calcula mediante:

$$\begin{aligned}x_2 &= x_1 \\ y_2 &= sh_y \cdot x_1 + y_1\end{aligned}$$

Donde sh_y es el factor de deformación respectivo. Su representación matricial es:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ sh_y & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

De igual manera, los puntos situados sobre el eje y ($x = 0$) permanecen invariantes.

3. DESARROLLO DE LA PRÁCTICA

Para calcular la deformación directamente en el hardware gráfico, enviando la matriz correspondiente como un parámetro `uniform`.

3.1. Código Fuente Completo en C++ con Shaders

A continuación se presenta la arquitectura moderna para renderizar y deformar primitivas geométricas:

```

1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <iostream>
4 #include <glm/glm.hpp>
5 #include <glm/gtc/type_ptr.hpp>
6
7 // Código del Vertex Shader
8 const char* vertexShaderSource = R"glsl(
9     #version 330 core
10     layout (location = 0) in vec2 aPos;
11     uniform mat4 uTransform;
12     void main() {
13         gl_Position = uTransform * vec4(aPos, 0.0, 1.0);
14     }
15 )glsl";
16
17 // Código del Fragment Shader
18 const char* fragmentShaderSource = R"glsl(
19     #version 330 core
20     out vec4 FragColor;
21     uniform vec4 uColor;
22     void main() {
23         FragColor = uColor;
24     }
25 )glsl";
26
27 // Variables de control
28 unsigned int shaderProgram;
```

```

29 unsigned int VAO, VBO, EBO;
30
31 void setupShaders() {
32     // Compilar Vertex Shader
33     unsigned int vertexShader = glCreateShader(GL_VERTEX_SHADER);
34     glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
35     glCompileShader(vertexShader);
36
37     // Compilar Fragment Shader
38     unsigned int fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
39     glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
40     glCompileShader(fragmentShader);
41
42     // Linkar el Shader Program
43     shaderProgram = glCreateProgram();
44     glAttachShader(shaderProgram, vertexShader);
45     glAttachShader(shaderProgram, fragmentShader);
46     glLinkProgram(shaderProgram);
47
48     glDeleteShader(vertexShader);
49     glDeleteShader(fragmentShader);
50 }
51
52 void setupGeometry() {
53     // Definicion de un rectangulo normalizado [-0.5, 0.5]
54     float vertices[] = {
55         -0.25, -0.25, // Inferior Izquierda (0)
56         0.25, -0.25, // Inferior Derecha (1)
57         0.25, 0.25, // Superior Derecha (2)
58         -0.25, 0.25 // Superior Izquierda (3)
59     };
60     unsigned int indices[] = { 0, 1, 2, 3 };
61
62     glGenVertexArrays(1, &VAO);
63     glGenBuffers(1, &VBO);
64     glGenBuffers(1, &EBO);
65
66     glBindVertexArray(VAO);
67
68     glBindBuffer(GL_ARRAY_BUFFER, VBO);
69     glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
70
71     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
72     glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
73     GL_STATIC_DRAW);
74
75     glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(float), (void*)0);
76     glEnableVertexAttribArray(0);
77 }
78
79 int main() {
80     if (!glfwInit()) return -1;
81     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
82     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
83     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
84
85     GLFWwindow* window = glfwCreateWindow(600, 600, "Deformacion 2D - Modern
86     OpenGL", NULL, NULL);
87     if (!window) { glfwTerminate(); return -1; }
88     glfwMakeContextCurrent(window);
89     glewInit();

```

```

89     setupShaders();
90     setupGeometry();
91
92     // Factores de Deformacion (Modificar segun se requiera)
93     float shX = 1.5;
94     float shY = 0.0;
95
96     while (!glfwWindowShouldClose(window)) {
97         glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
98         glClear(GL_COLOR_BUFFER_BIT);
99
100        glUseProgram(shaderProgram);
101
102        // Matriz de deformacion en formato de columna (Column-major order)
103        glm::mat4 transform = glm::mat4(1.0f);
104        transform[1][0] = shX; // Fila 0, Columna 1 -> Representa el corte en X
105        transform[0][1] = shY; // Fila 1, Columna 0 -> Representa el corte en Y
106
107        int transformLoc = glGetUniformLocation(shaderProgram, "uTransform");
108        glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform))
109    ;
110
111    // Dibujar contorno del objeto original/deformado
112    int colorLoc = glGetUniformLocation(shaderProgram, "uColor");
113    glUniform4f(colorLoc, 1.0f, 0.0f, 0.0f, 1.0f); // Color Rojo
114
115    glBindVertexArray(VAO);
116    glDrawElements(GL_LINE_LOOP, 4, GL_UNSIGNED_INT, 0);
117
118    glfwSwapBuffers(window);
119    glfwPollEvents();
120
121    }
122
123    glDeleteVertexArrays(1, &VAO);
124    glDeleteBuffers(1, &VBO);
125    glDeleteBuffers(1, &EBO);
126    return 0;
127 }

```

Listing 1: Implementación en Modern OpenGL para Shearing 2D

4. TAREA

4.1. Enunciado

Escribir un programa en Modern OpenGL que deforme la siguiente figura geométrica (Letra 'E') aplicando los factores de deformación (Shearing) de $sh_x = 2$ y $sh_y = 3$.

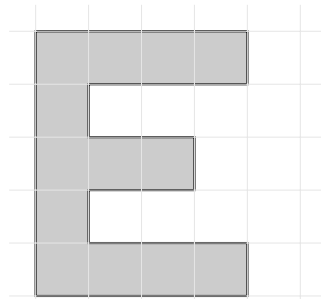


Figura original: Letra “E”

Indicación para el desarrollo: Deberás cambiar el arreglo de vértices de la geometría (`vertices[]`) e índices (`indices[]`) en el programa base de C++ para mapear los 12 puntos que componen el polígono de la letra “E” y renderizarlos adecuadamente usando `GL_TRIANGLES` (mediante triangulación manual) o `GL_LINE_LOOP` para el contorno.

5. BIBLIOGRAFÍA

- Donald Hearn, M. Pauline Baker (2006); *Gráficas por Computadora con OpenGL*; Pearson Prentice Hall; 3° edición; Madrid.
- Rimón Elias (2019); *Digital Media: A Problem-solving Approach for Computer Graphics*; eBook. New York.
- Sumanta Guha (2019); *Computer Graphics Through OpenGL*; Third Edition; Taylor & Francis Group.
- Gordon V. Scott, John Clevenger (2019); *Computer Graphics Programming in OpenGL with C++*; Mercury Learning and Information.